

ЛАБОРАТОРНАЯ РАБОТА

по дисциплине: «Технологии баз данных»

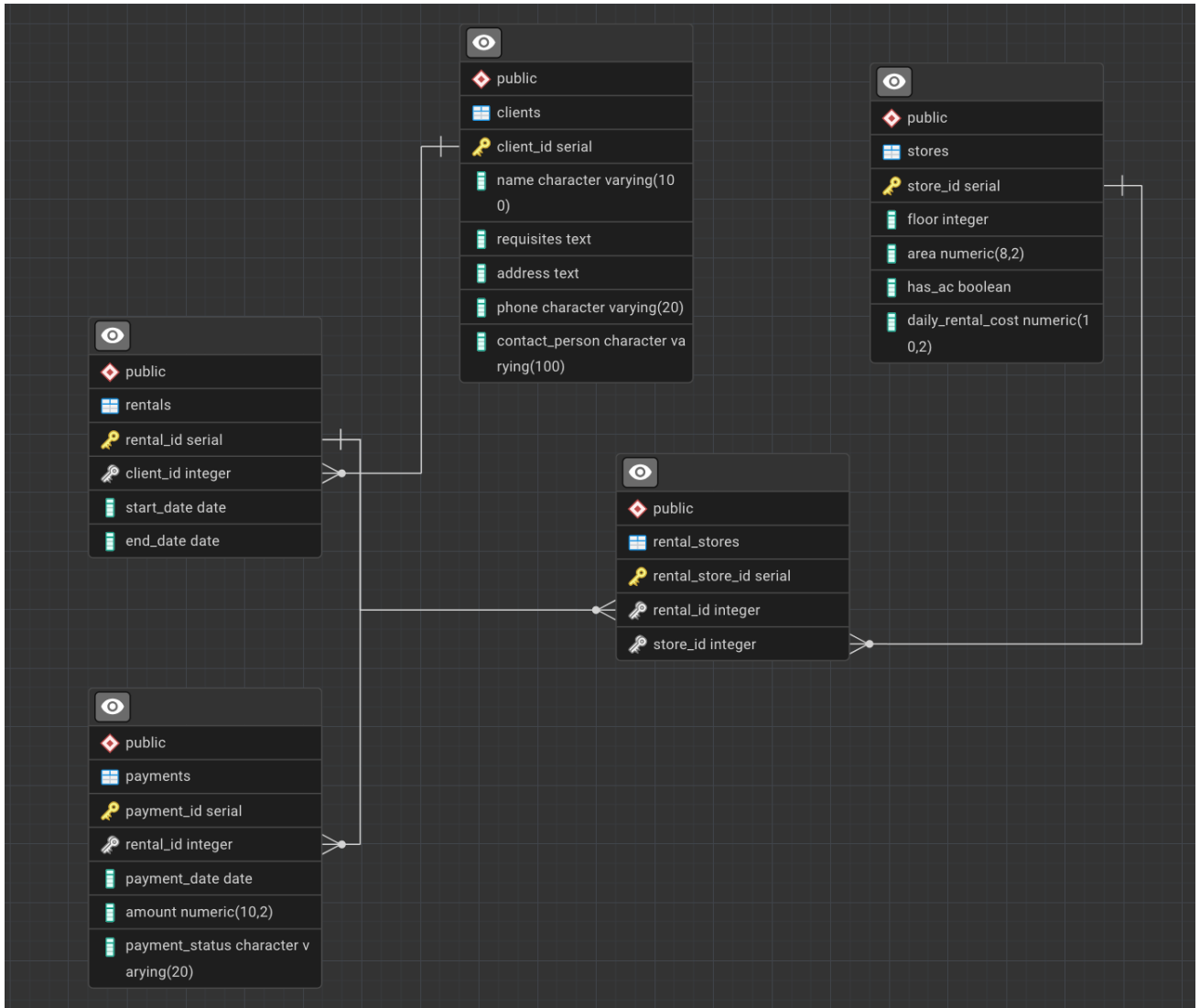
на тему:

**«Проектирование и реализация базы данных
«Аренда торговых площадей»»**

Выполнила:
Студентка 2 курса
Черкасова Полина Денисовна

1. Концептуальная схема предметной области

В торговом центре выделены торговые точки (stores). Клиенты (clients) арендуют точки на определённый срок (rentals). Один клиент может арендовать несколько точек. По аренде фиксируются ежемесячные платежи (payments).



2. Структура базы данных (SQL-скрипт)

Таблица торговых точек

```
CREATE TABLE stores (  
    store_id SERIAL PRIMARY KEY,  
    floor INTEGER NOT NULL CHECK (floor > 0),  
    area NUMERIC(8,2) NOT NULL CHECK (area > 0),  
    has_ac BOOLEAN DEFAULT FALSE,  
    daily_rental_cost NUMERIC(10,2) NOT NULL CHECK (daily_rental_cost > 0)  
);
```

	store_id [PK] integer	floor integer	area numeric (8,2)	has_ac boolean	daily_rental_cost numeric (10,2)
1	1	1	35.50	true	5000.00
2	2	2	42.00	true	6200.00
3	3	1	18.75	false	3000.00
4	4	3	55.20	true	7500.00
5	5	2	27.40	false	4100.00
6	6	1	95.00	true	12000.00
7	7	2	42.50	false	5500.00
8	8	3	200.00	true	25000.00
9	9	1	15.00	false	2800.00
10	10	4	310.00	true	45000.00

Таблица клиентов

```
CREATE TABLE clients (
    client_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    requisites TEXT,
    address TEXT,
    phone VARCHAR(20),
    contact_person VARCHAR(100)
);
```

	client_id [PK] integer	name character varying (100)	requisites text	address text	phone character varying (20)	contact_person character varying (100)
1	1	Nike	123456789	Moscow	+79991112233	Ivan Petrov
2	2	Adidas	987654321	Saint Petersbu...	+79994445566	Anna Smirnova
3	3	Zara	456123789	Kazan	+79997778899	Olga Ivanova
4	4	Puma	111222333	Novosibirsk	+79001112233	Alexey Kozlov
5	5	Reebok	444555666	Ekaterinburg	+79004445566	Elena Morozova
6	6	H&M	777888999	Samara	+79007778899	Dmitry Volkov
7	7	Uniqlo	000111222	Rostov-on-Don	+79001110022	Maria Sokolova
8	8	Decathlon	333444555	Krasnodar	+79003334455	Pavel Lebedev

Таблица аренды

```
CREATE TABLE rentals (
    rental_id SERIAL PRIMARY KEY,
    client_id INTEGER NOT NULL REFERENCES clients(client_id),
    start_date DATE NOT NULL,
    end_date DATE NOT NULL
);
```

	rental_id [PK] integer	client_id integer	start_date date	end_date date
1	1	1	2025-01-01	2025-12-31
2	2	2	2025-02-01	2025-08-31
3	3	3	2025-03-15	2025-11-15
4	4	4	2025-04-01	2025-10-01
5	5	5	2025-05-15	2025-12-15
6	6	1	2025-06-01	2025-09-30
7	7	2	2025-07-10	2026-01-10
8	8	3	2025-08-01	2025-11-30

Связь аренды с точками (многие ко многим)

```
CREATE TABLE rental_stores (
    rental_store_id SERIAL PRIMARY KEY,
    rental_id INTEGER NOT NULL REFERENCES rentals(rental_id),
    store_id INTEGER NOT NULL REFERENCES stores(store_id)
);
```

	rental_store_id [PK] integer	rental_id integer	store_id integer
1	1	1	1
2	2	1	2
3	3	2	3
4	4	3	4
5	5	3	5
6	6	4	1
7	7	4	3
8	8	5	2
9	9	6	5
10	10	6	6
11	11	6	7
12	12	7	4
13	13	8	8
14	14	8	9

Таблица платежей

```
CREATE TABLE payments (  
    payment_id SERIAL PRIMARY KEY,  
    rental_id INTEGER NOT NULL REFERENCES rentals(rental_id),  
    payment_date DATE NOT NULL,  
    amount NUMERIC(10,2) NOT NULL CHECK (amount > 0),  
    payment_status VARCHAR(20) NOT NULL  
);
```

	payment_id [PK] integer	rental_id integer	payment_date date	amount numeric (10,2)	payment_status character varying (20)
1	1	1	2025-01-10	150000.00	paid
2	2	1	2025-02-10	150000.00	paid
3	3	2	2025-02-15	90000.00	pending
4	4	3	2025-03-20	120000.00	paid
5	5	1	2025-03-10	150000.00	paid
6	6	1	2025-04-10	150000.00	paid
7	7	2	2025-03-15	80000.00	paid
8	8	2	2025-04-15	80000.00	pending
9	9	3	2025-04-20	60000.00	paid
10	10	3	2025-05-20	60000.00	paid
11	11	4	2025-04-05	180000.00	paid
12	12	5	2025-05-20	75000.00	pending
13	13	6	2025-06-10	200000.00	paid
14	14	7	2025-07-15	95000.00	paid

3. Агрегатные запросы

За. Соединение таблиц (JOIN)

Код:

```
SELECT  
    c.name AS client_name,  
    COUNT(p.payment_id) AS payment_count,  
    SUM(p.amount) AS total_paid  
FROM clients c  
JOIN rentals r ON c.client_id = r.client_id  
JOIN payments p ON r.rental_id = p.rental_id  
GROUP BY c.client_id  
ORDER BY total_paid DESC;
```

Результат:

	client_name character varying (100) 🔒	payment_count bigint 🔒	total_paid numeric 🔒
1	Nike	5	800000.00
2	Adidas	4	345000.00
3	Zara	3	240000.00
4	Puma	1	180000.00
5	Reebok	1	75000.00

3b. Функции даты/времени

Код:

```
SELECT
    rental_id,
    start_date,
    end_date,
    AGE(end_date, start_date) AS rental_duration,
    EXTRACT(YEAR FROM start_date) AS start_year,
    TO_CHAR(end_date, 'Month') AS end_month_name
FROM rentals
WHERE EXTRACT(MONTH FROM start_date) BETWEEN 3 AND 6;
```

Результат:

	rental_id [PK] integer ✎	start_date date ✎	end_date date ✎	rental_duration interval 🔒	start_year numeric 🔒	end_month_name text 🔒
1	3	2025-03-15	2025-11-15	8 mons	2025	November
2	4	2025-04-01	2025-10-01	6 mons	2025	October
3	5	2025-05-15	2025-12-15	7 mons	2025	December
4	6	2025-06-01	2025-09-30	3 mons 29 days	2025	September

3с. Текстовые функции

Код:

```
SELECT
    name,
    LENGTH(name) AS name_length,
    UPPER(name) AS name_upper,
    POSITION('o' IN LOWER(name)) AS first_o_position
```

```
FROM clients
WHERE name ILIKE '%o%';
```

Результат:

	name character varying (100) 🔒	name_length integer 🔒	name_upper text 🔒	first_o_position integer 🔒
1	Reebok	6	REEBOK	5
2	Uniqlo	6	UNIQLO	6
3	Decathlon	9	DECATHLON	8

3d. Родители без дочерних записей

Код:

```
SELECT
    c.client_id,
    c.name
FROM clients c
LEFT JOIN rentals r ON c.client_id = r.client_id
WHERE r.rental_id IS NULL;
```

Результат:

	client_id [PK] integer ✎	name character varying (100) ✎
1	8	Decathlon
2	6	H&M
3	7	Uniqlo

4. Триггер INSERT

Условие: аренда не может быть короче 7 дней.

```
CREATE OR REPLACE FUNCTION check_rental_min_duration()
RETURNS TRIGGER AS $$
BEGIN
    IF (NEW.end_date - NEW.start_date) < 7 THEN
        RAISE EXCEPTION 'Аренда должна быть не менее 7 дней';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_check_rental_duration
BEFORE INSERT ON rentals
FOR EACH ROW
EXECUTE FUNCTION check_rental_min_duration();
```

Проверка:

```
INSERT INTO rentals (client_id, start_date, end_date)
VALUES (1, '2025-06-01', '2025-06-05');
```

Ожидаемая ошибка:

ERROR: Аренда должна быть не менее 7 дней

Результат:

```
Data Output  Messages  Notifications
ERROR:  Аренда должна быть не менее 7 дней
CONTEXT:  PL/pgSQL function check_rental_min_duration() line 4 at RAISE

SQL state: P0001
```

5. Процедура с курсором

Задача: повысить стоимость аренды для точек без кондиционера.

```
CREATE OR REPLACE PROCEDURE increase_rent_for_no_ac(percent_increase NUMERIC)
LANGUAGE plpgsql AS $$
DECLARE
    cur CURSOR FOR
        SELECT store_id, daily_rental_cost
        FROM stores
        WHERE has_ac = FALSE;
    store_rec RECORD;
BEGIN
    OPEN cur;
    LOOP
        FETCH cur INTO store_rec;
        EXIT WHEN NOT FOUND;

        UPDATE stores
        SET daily_rental_cost = daily_rental_cost * (1 + percent_increase/100)
        WHERE store_id = store_rec.store_id;
    END LOOP;
    CLOSE cur;
END;
$$;
```


ВЫЗОВ:

```
CALL increase_rent_for_no_ac(10);
```

	store_id [PK] integer	has_ac boolean	cost_before numeric (10,2)	cost_after numeric
1	3	false	3300.00	3000.00
2	5	false	4510.00	4100.00
3	7	false	6050.00	5500.00
4	9	false	3080.00	2800.00

6. Удаление родителя без CASCADE

```
CREATE OR REPLACE PROCEDURE delete_client_with_rentals(p_client_id INTEGER)
LANGUAGE plpgsql AS $$
BEGIN
    DELETE FROM payments WHERE rental_id IN (
        SELECT rental_id FROM rentals WHERE client_id = p_client_id
    );
    DELETE FROM rental_stores WHERE rental_id IN (
        SELECT rental_id FROM rentals WHERE client_id = p_client_id
    );
    DELETE FROM rentals WHERE client_id = p_client_id;
    DELETE FROM clients WHERE client_id = p_client_id;
END;
$$;
```

Данные до удаления(для client_id = 2)

	client_id [PK] integer	name character varying (100)	requisites text	address text	phone character varying (20)	contact_person character varying (100)
1	2	Adidas	987654321	Saint Petersbu...	+79994445566	Anna Smirnova

	rental_id [PK] integer	client_id integer	start_date date	end_date date
1	2	2	2025-02-01	2025-08-31
2	7	2	2025-07-10	2026-01-10

Данные после удаления

```
1 SELECT * FROM clients WHERE client_id = 2;
```

Data Output Messages Notifications

	client_id [PK] integer	name character varying (100)	requisites text	address text	phone character varying (20)	contact_person character varying (100)
--	---------------------------	---------------------------------	--------------------	-----------------	---------------------------------	---

```
1 SELECT * FROM rentals WHERE client_id = 2;
2
```

Data Output Messages Notifications

	rental_id [PK] integer	client_id integer	start_date date	end_date date
--	---------------------------	----------------------	--------------------	------------------

7. Star-схема необязательных атрибутов

```
CREATE TABLE client_extra_attrs (
    client_id INTEGER REFERENCES clients(client_id) ON DELETE CASCADE,
    attr_name VARCHAR(100),
    attr_value TEXT,
    PRIMARY KEY (client_id, attr_name)
);
```

-- Добавляем атрибуты для трёх клиентов

```
INSERT INTO client_extra_attrs (client_id, attr_name, attr_value) VALUES
(1, 'discount_level', 'gold'),
(1, 'tax_number', '7712345678'),
(7, 'discount_level', 'silver'),
(8, 'discount_level', 'platinum'),
(3, 'loyalty_years', '5'),
(3, 'preferred_store', '2');
```

Результат:

1 `SELECT * FROM client_extra_attrs ORDER BY client_id, attr_name;`

Data Output Messages Notifications

SQL

	client_id [PK] integer	attr_name [PK] character varying (100)	attr_value text
1	1	discount_level	gold
2	1	tax_number	77123456...
3	3	loyalty_years	5
4	3	preferred_store	2
5	7	discount_level	silver
6	8	discount_level	platinum

8. Аналитический набор + рейтинг

```
CREATE OR REPLACE VIEW store_rental_stats AS
```

```
SELECT
```

```
    s.store_id,
```

```
    s.floor,
```

```
    s.area,
```

```
    s.daily_rental_cost,
```

```
    COUNT(DISTINCT rs.rental_id) AS total_rentals,
```

```
    COALESCE(SUM(p.amount), 0) AS total_revenue
```

```
FROM stores s
```

```
LEFT JOIN rental_stores rs ON s.store_id = rs.store_id
```

```
LEFT JOIN rentals r ON rs.rental_id = r.rental_id
```

```
LEFT JOIN payments p ON r.rental_id = p.rental_id
```

```
GROUP BY s.store_id;
```

Результат:

```
SELECT
    store_id,
    total_revenue,
    RANK() OVER (ORDER BY total_revenue DESC) AS revenue_rank,
    total_revenue - LEAD(total_revenue) OVER (ORDER BY total_revenue DESC) AS lead_gap
FROM store_rental_stats;
```

	store_id integer	total_revenue numeric	revenue_rank bigint	lead_gap numeric
1	1	780000.00	1	105000.00
2	2	675000.00	2	235000.00
3	5	440000.00	3	200000.00
4	4	240000.00	4	40000.00
5	6	200000.00	5	0.00
6	7	200000.00	5	20000.00
7	3	180000.00	7	180000.00
8	8	0	8	0
9	10	0	8	0
10	9	0	8	[null]

Заключение

В ходе работы спроектирована и реализована база данных «Аренда торговых площадей», выполнены все пункты задания: от концептуальной схемы до аналитических функций.